

P2 — Architecture Design (First Deliverable)

Historical Conversational AI · Big Data Management

1. Where we left off and where we're going

In P1 we built a fully containerized ingestion pipeline that pulls raw data about nine historical figures (philosophers, scientists, and authors) from seven different sources into a MinIO-based landing zone. Everything is orchestrated by Airflow and driven by a single registry file, so adding a new figure only requires one edit. Kafka handles a simulated hot path of real-time character mentions.

For P2 we need to clean that data (Trusted Zone), organize and enrich it for analysis (Exploitation Zone), and actually use it in downstream tasks (Consumption). The end goal hasn't changed: we want to power a system that can generate realistic, podcast-style interviews with historical figures who react to today's news. That goal shapes every design decision below — the stores we pick, the transformations we apply, and the consumption tasks we build all serve that vision.

After reviewing the two implementation options in the project statement, we chose the second one — using specialized databases for each data shape rather than keeping everything inside MinIO. The reasoning is straightforward: our landing zone contains structured tabular records, nested JSON documents, long-form book texts, audio files, images, and streaming events. Forcing all of that through a single object store would work, but it would mean we never learn the tools that actually fit each data type. Since we're in an academic setting, we'd rather build a more interesting pipeline and get hands-on experience with multiple technologies, even if it's more work.

2. Technology choices

We settled on four storage systems, each justified by what actually reads from it downstream.

DuckDB handles our structured analytical data — the philosopher metadata catalog, news articles, Stack Exchange Q&A pairs, and the streaming mention aggregates. We picked DuckDB over ClickHouse because our data volumes are tiny (thousands of rows, not billions), and DuckDB is an embedded file-based database that needs zero server infrastructure. It gives us fast columnar analytics with standard SQL, which is all we need for the dashboard KPIs and the star schema we'll build in the Exploitation Zone.

MongoDB handles our semi-structured documents — Wikipedia summaries, Wikiquote responses, and Gutenberg book catalogs. These all have nested, variable-shape JSON that doesn't flatten cleanly into a fixed schema. MongoDB lets us store them as-is, query nested fields, and group documents by figure. We considered just using DuckDB's JSON support, but MongoDB's document model is a much more natural fit when the schema varies between records.

Milvus is our vector database. It stores embeddings of every text chunk in the pipeline — paragraphs from Gutenberg books, Wikipedia summaries, Wikiquote quotes, and Stack Exchange answers, all embedded with the same model into the same collection. This is what makes the RAG chatbot possible: when a user asks Kant a question, we embed the question and find the most semantically similar chunks from Kant's corpus. We could have used FAISS as a simpler alternative, but Milvus gives us proper metadata filtering (retrieve only chunks belonging to a specific figure) and persistent storage, which matters once we have thousands of chunks across nine figures.

MinIO continues to hold raw unstructured assets that don't benefit from a specialized store — cleaned book texts, podcast audio, and portrait images. We extend it with two new buckets (**trusted-zone** and **exploitation-zone**) to separate the pipeline stages.

For transformations, we're using **Apache Spark** everywhere — both batch (Landing → Trusted → Exploitation) and streaming (Kafka → aggregated mention counts). Spark is overkill for our data volume, and we know that. We chose it because the project statement asks us to learn it, and because writing transformations in Spark means they'd scale to terabytes without a rewrite. We use Spark Structured Streaming for the hot path rather than Kafka Streams (which requires Java) or Flink (which would introduce a whole new runtime). Since Spark already handles our batch work, using it for streaming too keeps things consolidated.

Airflow continues to orchestrate the daily batch pipeline, extended with new tasks for the Trusted and Exploitation zones. The streaming job runs as a separate long-lived Docker service, not as an Airflow task — that's the standard pattern for streaming workloads.

3. Trusted Zone

The Trusted Zone applies information-preserving cleaning only — we fix representational issues without encoding assumptions about downstream tasks. The guiding question from the project statement is useful here: if two different downstream tasks consumed this data, would both benefit from this transformation? If yes, it belongs in the Trusted Zone. If one task would be harmed, it belongs closer to consumption.

Our structured sources (Philosophers API records, GNews articles, Stack Exchange Q&A) go through type casting, deduplication, schema flattening, and null normalization, then land as clean tables in a DuckDB file (**trusted.duckdb**). For example, Stack Exchange questions get their nested **owner** field

flattened, Unix timestamps cast to proper dates, and questions without accepted answers dropped since we only use Q+A pairs downstream.

The semi-structured documents (Wikipedia, Wikiquote, Gutenberg catalogs) get field name unification across sources (`extract` becomes `summary_text` everywhere), missing-field defaults, a consistent `figure_slug` and `domain` tag for filtering, and structural validation. Wikiquote responses in particular need unwrapping from the MediaWiki API envelope and HTML parsing to extract individual quote strings. All of this lands in MongoDB collections under a `trusted_db` database.

Unstructured assets get the lightest touch. Gutenberg book texts have the Project Gutenberg boilerplate headers and footers stripped and encoding normalized to UTF-8. Podcast audio files get validated for integrity (corrupt downloads removed) with duration and bitrate metadata extracted into sidecar JSON files. Portrait images get resized to a maximum of 800px on the longest edge and converted to a consistent JPEG format. All of this stays in MinIO under a `trusted-zone` bucket, organized by type and figure.

For the streaming path, a long-running Spark Structured Streaming job reads the `character-mentions` Kafka topic, applies a 1-minute tumbling window with a 5-minute watermark, and computes mention counts and average sentiment per figure per window. The output goes to a Delta table in MinIO.

Every transformation runs as a PySpark job invoked from Airflow. Each job does a full overwrite of its target, so re-running a failed task is always safe.

4. Exploitation Zone

The Exploitation Zone is where we organize data for actual use and compute new derived assets that would be expensive to regenerate on every query.

The centerpiece of the structured layer is a small **star schema in DuckDB** (`exploit.duckdb`). A `dim_figure` dimension table holds one row per historical figure with unified attributes pulled from both the Philosophers API and Wikipedia — name, domain, school of thought, birth/death years, biography summary, and a link to the canonical portrait. Around it sit fact tables: `fact_works` (one row per Gutenberg book per figure), `fact_quotes` (one row per Wikiquote quote), `fact_se_qa` (Stack Exchange questions linked to figures by tag matching), `fact_news_articles` (the full news article history, used by the chatbot's Interviewer agent), and `fact_mentions_1m` (the streaming aggregates from the hot path). The denormalization is deliberate — the dashboard can answer "how many works per figure, filtered by domain" with a single small join instead of chasing four separate tables.

In MongoDB we build a curated `figure_profiles` collection — one document per figure that unifies their biography, quotes, Stack Exchange questions, works list, academic links, and portrait URL into a single entity. This is what the dashboard's detail page and the chatbot's identity card both load in a single read.

The most technically interesting new asset is the **Milvus vector index**. We build a single collection called `corpus_chunks` that holds embeddings for every textual source in the pipeline, not just Gutenberg books. The embedding job reads cleaned books from MinIO, Wikipedia summaries and Wikiquote quotes from MongoDB, and Stack Exchange Q&A from DuckDB. Each source gets chunked appropriately — books into ~500-token chunks with paragraph-boundary overlap, Wikipedia summaries into ~300-token chunks, quotes and Q&A pairs as individual chunks. Everything gets embedded with `sentence-transformers/all-MiniLM-L6-v2` (a small model that runs on CPU) and upserted into Milvus with metadata tags for `figure_slug`, `source`, `book_id`, and `chunk_text`. The collection serves two access patterns: semantic vector search for the chatbot's Reasoner agent, and metadata-only sampling for the Voice agent's style exemplars. Approximate scale is around 10,500 chunks total across nine figures.

Finally, MinIO's `exploitation-zone` bucket holds one canonical portrait per figure (selected deterministically from the trusted set) and, as a stretch goal, Whisper transcripts of podcast audio.

5. Consumption tasks

We committed to three consumption tasks, each designed to exercise a different part of the architecture so that every storage choice is justified by what actually reads from it.

Dashboard

A multi-page Streamlit app. The overview page shows KPI cards (total figures, works, quotes, Q&A pairs), a bar chart of works per figure from DuckDB, and a news article timeline. Clicking a figure opens a detail page that loads the portrait from MinIO and the full profile document from MongoDB. The dashboard reads from three of our four stores and is the simplest task to build — roughly a day and a half of work.

Three-agent chatbot

This is the marquee feature and the one that directly delivers the project's stated end goal. Rather than a single LLM call, we split the chatbot into three focused agents, each with a single objective. The separation produces better output and makes the system debuggable — if something goes wrong, we can tell which agent failed.

The **Interviewer** is only active in interview mode. It reads recent news from DuckDB, and given a selected figure, generates 2–3 candidate questions designed to provoke that figure's philosophical perspective on current events. The user picks or edits a question before it goes forward. This keeps the user in control even when the generated questions aren't great.

The **Reasoner** is always active and is the RAG core. It takes a question (either from the Interviewer or typed directly by the user), embeds it, performs a top-k semantic search against Milvus filtered by the

selected figure's slug, and loads the figure's profile document from MongoDB as a stable identity card. It then generates a substantive, accurate answer in plain modern English — no stylization, just content faithful to the retrieved passages.

The **Voice** agent takes the Reasoner's plain-text output and rewrites it in the figure's literary style. To ground the stylistic transformation, it pulls 5 random paragraphs from the figure's actual writing via a metadata-only Milvus query (filtering by `figure_slug` and `source == "gutenberg"`). These exemplars go into the prompt alongside instructions to match the cadence, vocabulary, and rhetorical moves of the samples.

In free-form mode the flow is simply user question → Reasoner → Voice → display. In interview mode it's news article → Interviewer → user picks a question → Reasoner → Voice → display. The UI shows the voiced response by default, with a toggle to reveal the Reasoner's intermediate plain-text answer and an expandable panel showing the retrieved Milvus chunks as citations.

This single feature exercises all four storage technologies: DuckDB (news for the Interviewer), MongoDB (figure profiles for the Reasoner), Milvus (semantic retrieval for the Reasoner, style sampling for the Voice), and the Anthropic API for generation at each agent step.

Real-time mention monitor

A Streamlit page that auto-refreshes every 5 seconds, reads the latest 60 minutes of `fact_mentions_1m` from DuckDB, and plots a live line chart of mentions per minute per figure, a sentiment heatmap, and a "trending now" card. This is the only consumption task that reads from the streaming pipeline output, and without it the entire Spark Structured Streaming work would be a system that ingests but is never read. Build effort is about half a day once the dashboard skeleton exists.

6. Orchestration, containerization, and build plan

The existing Airflow DAG gets extended with new task groups: trusted-zone cleaning (five parallel Spark jobs, one per data shape), exploitation-zone curation (star schema build, figure profiles, Milvus embedding, portrait selection), and a small batch task that moves streaming aggregates into DuckDB. The Spark Structured Streaming job runs as a separate long-lived Docker service outside of Airflow.

Docker Compose grows from 8 services to 14: we add MongoDB, Milvus standalone, a Spark master and worker, a Streamlit container for the consumption app, and a dedicated Spark Streaming container. The full stack still comes up with a single `docker compose up -d`.

For governance, we'll implement lineage tracking as our one mechanism. Every Spark job writes a small `_lineage.json` sidecar recording its timestamp, Airflow task ID, inputs, transformation description, and output path. Reading these sidecars backward traces any record from Exploitation back to its original API

response in the Landing Zone. We'll also define three formal data products (Figure Knowledge Pack, News Snapshot, Streaming Mention Trends) with a full data-product card for the first one.

The build plan across the remaining weeks is: Week 3 stands up the new infrastructure and wires empty Airflow tasks into the DAG topology. Week 4 fills in the Trusted Zone Spark jobs. Week 5 builds the Exploitation Zone, with the Milvus embedding pipeline last since it's the most complex. Week 6 builds the three consumption tasks, with the chatbot agents in dependency order — Reasoner first (end-to-end on Kant only as proof of life), then Voice, then Interviewer last. If time gets tight, the planned cuts in order are: Whisper transcripts, the Interviewer agent, reducing Milvus to a single figure's corpus, and finally the sentiment heatmap on the monitor page.